3D printer improvements

From the community for the community, mostly via heavily customized tools

Some links about me. Many of my 3D designs are free. I also post on Google+ and in another blog, oz4.us

Sticky posts

survival kit, openscad tutorial, material for artists, what cannot be 3D printed, oz4.us...

Tweets by tridimake

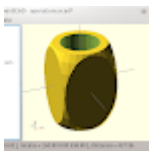
Popular posts

Replacing LinuxCNC with a Smoothieboard for CNC milling. Not as easy as for 3D printing!

I was running LinuxCNC for the last years and I wanted a smaller setup, with room for improvement. Since I had a Smoothieboard around, whi...

How to use Openscad (1), tricks and tips to design a parametric 3D object

Part 1/5: Introduction to constructive solid geometry with Openscad Some technical and non-technical people keep asking me how I create...

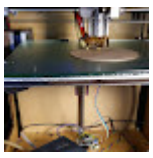


How to use Openscad (2): variables and modules for parametric designs

Part 2/5: Variable and parametric design The previous part addressed the basics of Openscad. It relied mostly on "immediate values...

How to use Openscad (4): children and advanced topics

Part 4/5: children, factorized placement and chained hulls Yes, it was made with Openscad and it is parametric! (extreme collaborativ...



Cooling 3D prints with an aquarium pump

3D printing cooled with an aquarium pump! It works well and it cannot be quieter. T he pump is bulky, but the overall is a very compact...

review (32) general (26) issue (16) mechanical (16) replacement (14) software (13) addon (12) bowden tube (10) business (10) consumables (10) heat (10) feeder (8) hot end (8) weird (7) motor (6) off-topic (6) trick (6) electric (4) bed (3) cnc (2)

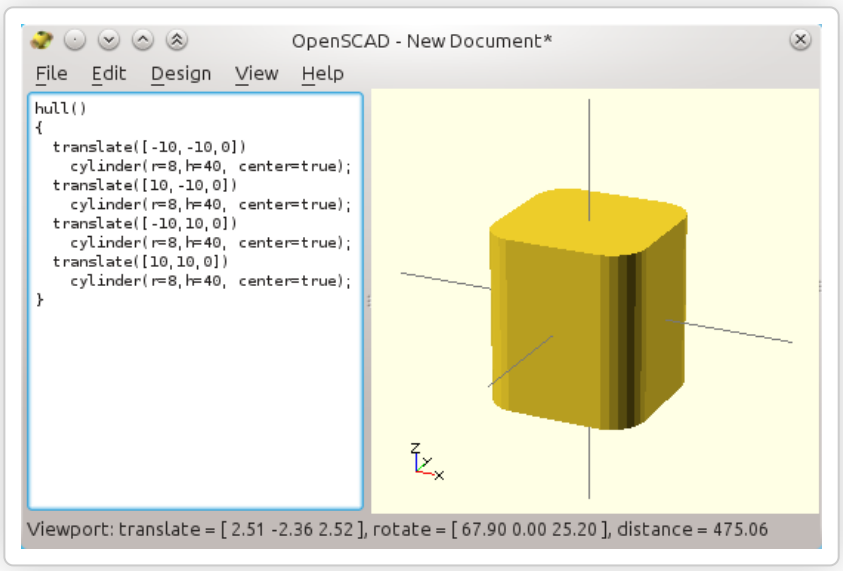
Saturday, November 22, 2014

How to use Openscad (3): iterations, extrusions and more modularity!

Part 3/5: iteration, extrusion and useful parametrized CSG techniques

Repeating shapes

As we saw in the previous article, repeating a shape by copy/pasting its Openscad definition is a bad practice. It increases the risk of mistakes just because of the slight changes that have to be made on each of the copies. And any "regularity" should be factorized: let the computer do our work!



The former way we built a (partially) rounded cube. Four copy/pastes? Boo!

See how the four columns really are all the same cylinder, where only the position changes? This is where we can and we should use **loops** instead. And once again there are different ways to do so.

This article is part of a longer serie:

1. Introduction to constructive solid geometry with OpenSCAD
2. Variables and modules for parametric designs
3. **Iteration, extrusion and useful parametrized CSG techniques**
4. Children, factorized placement and chained hulls

The "for(x = range) block;" repeats the block once with *x* set to each of the given values in the *range*, just like if Openscad was automatically copy/pasting the command block.

Ranges may be a list of values, separated by a comma, like this (one loop within another loop):

```
union()
{
  for(x=[-10,+10]) // repeat the following with two variants for x
  {
    for(y=[-10,+10]) // repeat again but this time for y
    {
      translate([x,y,0])
        cylinder(r=8,h=50, center=true);
```

```
      }
    }
    // two more shapes
    cube([20,20+2*8,50], center=true);
    cube([20+2*8,20,50], center=true);
  }
```

So what goes on here?

Openscad sets `x` to the first value of its range -10, then it does the same with `y` (to its first value, -10 also).
Then, within the inside *loop*, it does a translation to `(x,y)` and adds a cylinder there.
Once done, it *loops* with the second value of y (it sets `y` to +10), and adds another cylinder at (-10,10).
When it reaches the end of the y range, it *loops* on x next time, with `x`=+10 and so it does again a loop on `y`=-10 and again with `y`=+10.

Overall, the inner cylinder will be created once for each of the four locations! Hence the result looks like exactly like previously, but *without the repeated code*.

This writing is more compact, and it helps to convert the source code to a more compact and parametric version: indeed, the size and roundness can easily be made variable now, and be set once at the beginning.

I would write it this way, to minimize the presence of the *hardcoded* value "10". The idea really is to put 4 cylinders around the vertical axis at the same distance. This distance should be seen only once for more readability and easier parametrization.

See how the remaining of the code becomes both generic and more readable with these refinements?

```
// Parameters
size=20;
roundness=8;
height=50;
// Shapes
union()
{
  for(x=[-1,+1])
    for(y=[-1,+1])
      translate([x*size/2,y*size/2,0])
        cylinder(r=roundness,h=height, center=true);
  cube([size, size+2*roundness, height], center=true);
  cube([size + 2*roundness, size, height], center=true);
}
```

We already used vectors when specifying the three X,Y,Z arguments used to define cubes or translations for example. These are just vectors of 3 values. Note how they can be stored themselves in variables:

```
cube_size=[20,10,5]; // a triplet, aka 3-item vector stored
cube(cube_size); // ...in a variable, used to create a cube!
```

The "for" loops handle vectors of any size like `[v1,v2,v3,v4]`, but we can benefit from a few additional variations, in order to improve the 4-cylinder example further.

Less code almost always means less bugs: a *range* can be specified as a list of values as above, but it can also be defined by a `[start:step:stop]` triplet. This is done below, where `r` is set in turn to 0, then 90, and finally 180.
We stop "at" 359°, i.e. before reaching 360° as it would be at the exact same place as 0° (and hence it would create a second, useless, cylinder there).

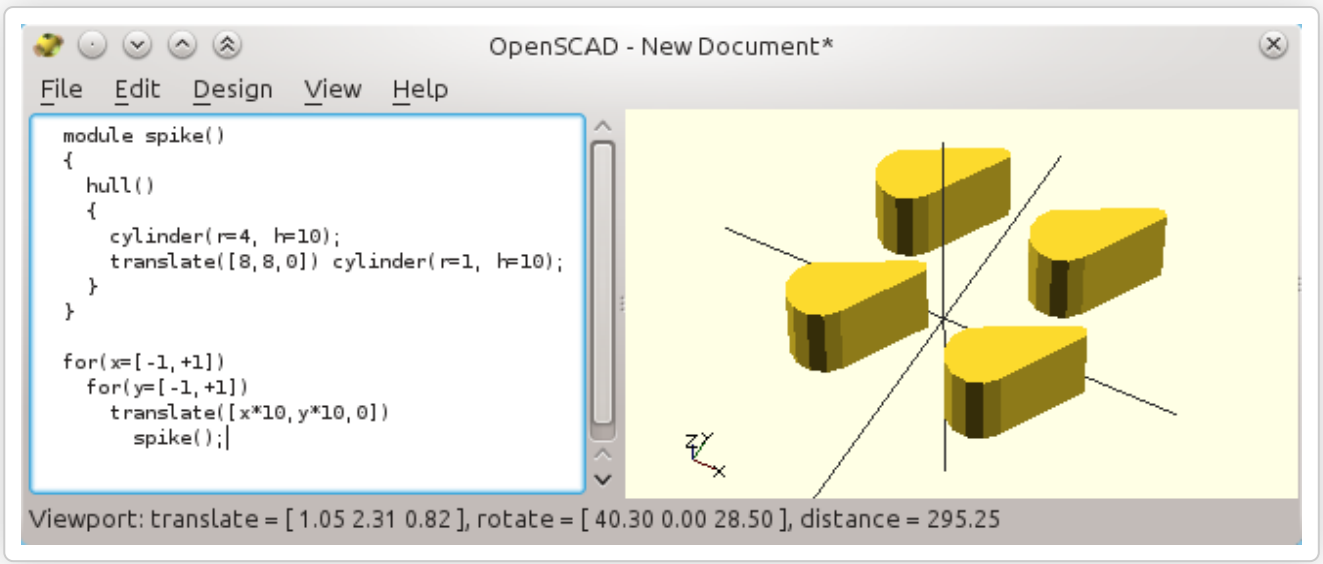
To do so, we use variable "r" to rotate around the Z axis below. Then, the column seemingly positioned at (10,10) is in fact "rotated" to each of the four symmetrical points around the (0,0) central axis.

```
size=20;
roundness=8;
height=50;
union()
{
  for(r=[0:90:359]) // stop *before* 360° (as 360°=0°)
    rotate([0,0,r])
      translate([size/2,size/2,0])
        cylinder(r=8,h=height, center=true);
  // same technique for the "inside" cuboids:
  for(r=[0,90])
    rotate([0,0,r])
      cube([size, size+2*roundness, height], center=true);

}
```

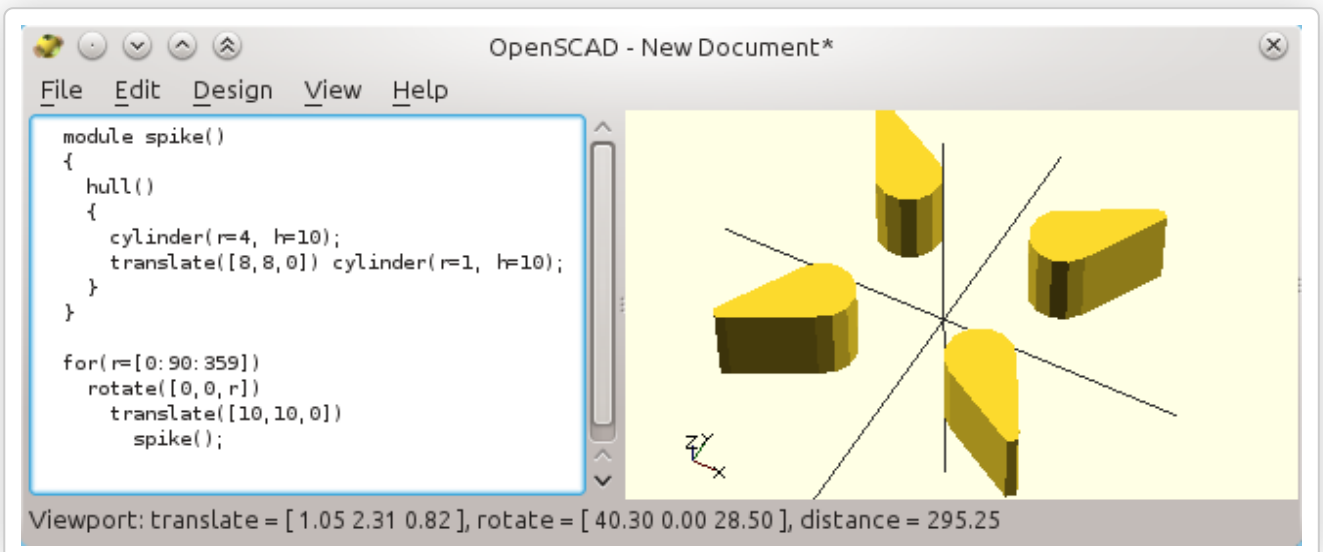
There are still other ways to create the same design (eg. by intersections, and so).

I tend to prefer either rotation, or inverted axes (with the scale operator and -1 on some axes). Actually, translating as we did above is probably not the wisest choice, because it does not change the "orientation" of the duplicated object

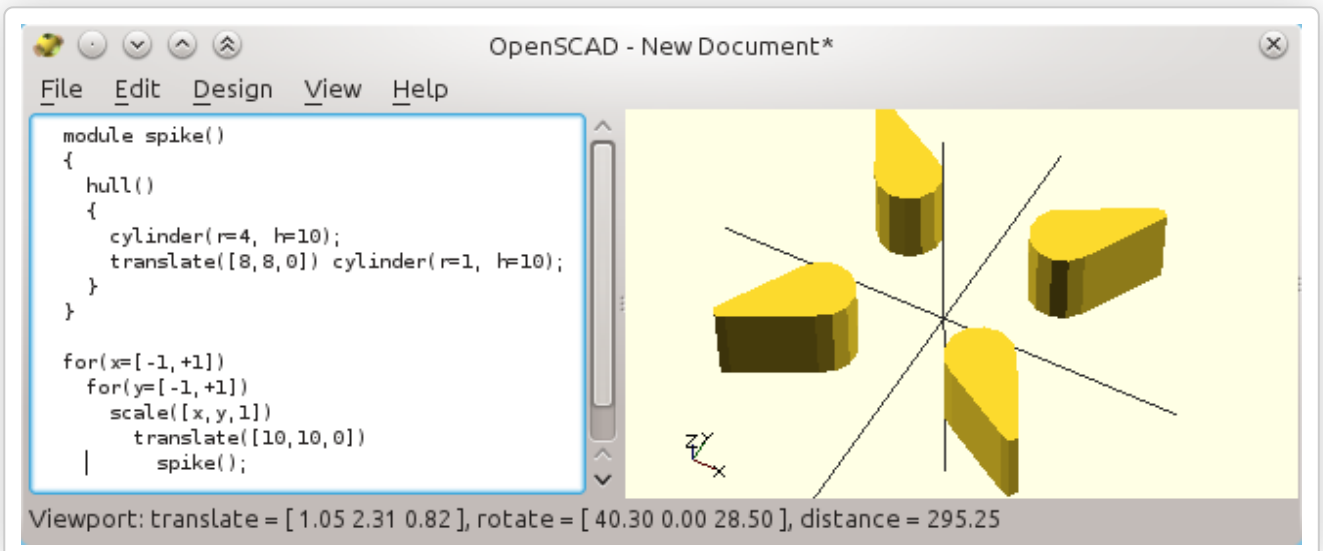
itself. No biggie for a rotation-invariant cylinder, but you see below the issue with a shape that is no more a cylinder.



Translations will keep the same orientation for the four parts!



Rotation is often preferred as it also "fixes" the orientation of the duplicated part.



But scaling operators also are useful to invert axes (click to see the code).

A note on the *scope* of the variable:

If you recall, we use curly brackets to group items together, so that a modifier like `translate` shifts more than only the objects that follows immediately.

You must also keep this in mind with `for` loops. The loop variable (like `r,x,y` above) are usable only within the so-called "scope" of the loop, which is "below" in the hierarchy. No problem when *only one operation* follows, as in the former cascade of loops.
But as soon as you need more than one command, you need to use braces:

```
// The following is OK:
for(t=[0:10:100])
  cylinder(r=t,h=t); // concentric cylinders depends on "t"

// But let us say you need to see "t" successive values
for(t=[0:10:100])
  echo(t); // print the value in the console
  cylinder(r=t,h=t); // wrong!

// It failed because openscad ignores blanks and indentation.
// So it read it as if you had written it like this:
for(t=[0:10:100])
  echo(t); // print the variable in the console
cylinder(r=t,h=t); // fails! "t" is unknown at this stage.

// Use curly brackets to specify the loop content:
for(t=[0:10:100])
```

```
{
  echo(t); // print the value in the console
  cylinder(r=t,h=t); // now OK !
  // "t" is usable only in the *scope* of the for loop!
}
```

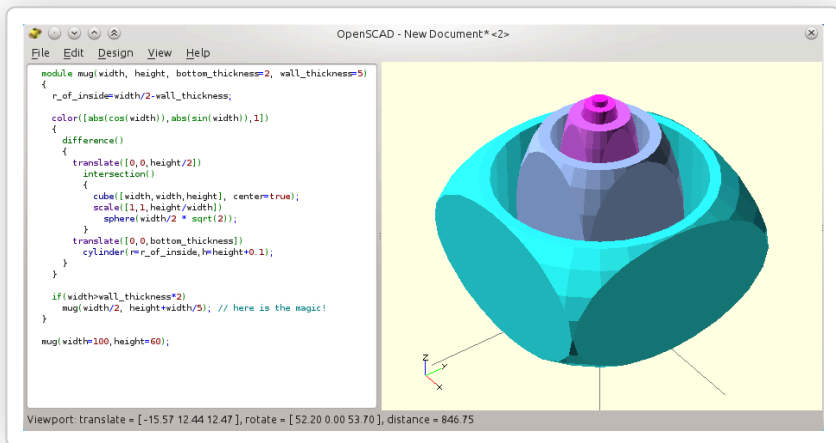
Some seasoned programmers use braces all the time, even when they are unneeded. It makes the source code a little longer, but it makes it more consistent. More importantly in my opinion, it lets you add statements later on, without the need to alter existing code -- like the `echo` above.

Recursive designs, and conditionals

Only the latest version of Openscad made recursion possible. Previously, variables had the amazing property that they were ... *constant*! Advanced operators like "assign( )" helped circumvent some of the issues, but they made the source code even worse in my opinion.

What is recursion? Well, it happens when a module calls *itself* in order to build more versions of its own shape, smaller or in another place. To avoid looping indefinitely (and killing the computer memory and CPU), we must also introduce conditionals. A *test* will be made to stop calling ourselves after some condition is met, e.g. like when the part becomes too small.

Remember the mug-with-a-cup that we made in part two? Here is how we can write a recursive mug, that will add smaller-mugs-in-mugs till it get smaller than the constant wall width. It would not make any sense to go "deeper" in the recursion anyhow.



A recursive mug. Make sure you have a version of Openscad that allows this
(an indicator is that the syntax is now colored: another long-waited for feature!)

By the way, you see also the use of the "color" operator, that highlights each of the sub-mugs. The color components were chosen according to the size of the mug, so we get an automatic color gradient whatever the depth of the recursion.

```
module mug(width, height, bottom_thickness=2, wall_thickness=5)
{
  r_of_inside=width/2-wall_thickness;
  echo(width); // print the value in the console

  color([abs(cos(width)),abs(sin(width)),1]) // color with size
  {
    difference()
    {
      translate([0,0,height/2])
        intersection()
        {
          cube([width,width,height], center=true);
          scale([1,1,height/width])
            sphere(width/2 * sqrt(2));
        }
      translate([0,0,bottom_thickness])
        cylinder(r=r_of_inside,h=height+0.1);
    }
  }
  // At this stage we are back to the default union() behavior
  if(width > wall_thickness*2) // only if there is room!
    mug(width/2, height+width/5); // add a smaller mug here
}

mug(width=100,height=60); // calling the top object
```

If you still get only one mug, check the console for an error like "WARNING: Ignoring recursive module instanciation of 'difference'.". This would tell that your version of Openscad does not support recursion and that you should probably upgrade. In the "Help / About" menu, check that the version is more recent than the 2014.01.29 (e.g. the 2014.04.02 as above).

Also the generated sizes are now harder to track, so we used the **echo** command to show how to get the value of width each time the module is called:

```
echo(width);
```

After you refresh the design, the console dumps the following:

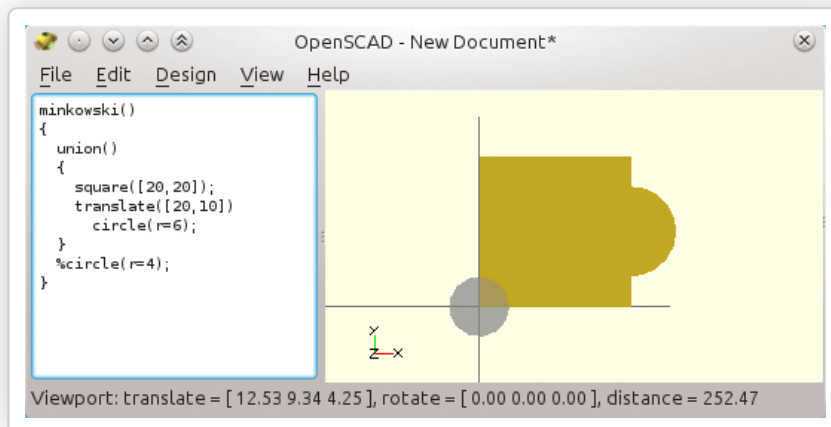
```
ECHO: 100
ECHO: 50
ECHO: 25
ECHO: 12.5

ECHO: 6.25
```

Important note: the very weird properties of *variables* in Openscad will not be discussed here. Even though recursion was recently allowed and not discussed in the following document, I highly suggest reading this detailed presentation of variables when you need it, written by +Stephanie Shaltes. By the way, she regularly posts about advanced subjects in Openscad so she may be interesting to follow on G+.

Advanced concepts

The **minkowski** operator is quite cool. But it is so slow that we almost never use it. The effect is like when a shape is used to paint or "brush" a parent shape: e.g. a small sphere will "round" all the edges and surfaces of a given reference object.

Shown below is an example where we use only 2D shapes, that Openscad also supports (we then use `square` in place of `cube`, `circle` for `spheres` and so).



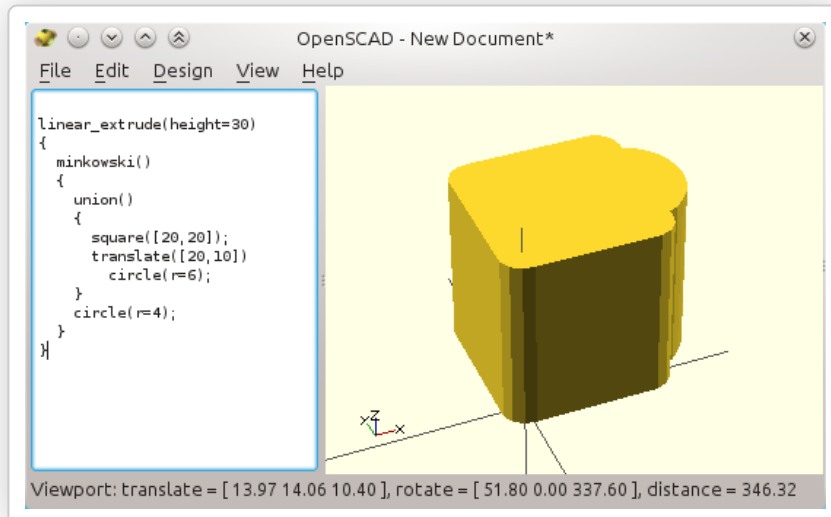
The minkowski operator, illustrated here on 2D Openscad shapes.
It does works in 3D also but is so slow that the shape is usually better designed otherwise.

More importantly, there are also two kinds of **extrusions**, linear and circular, that makes 3D shapes out of 2D shapes.

They are very useful to create 3D shapes out of "industrial" 2D designs (often based on the Autocad DXF file format that Openscad knows how to import).

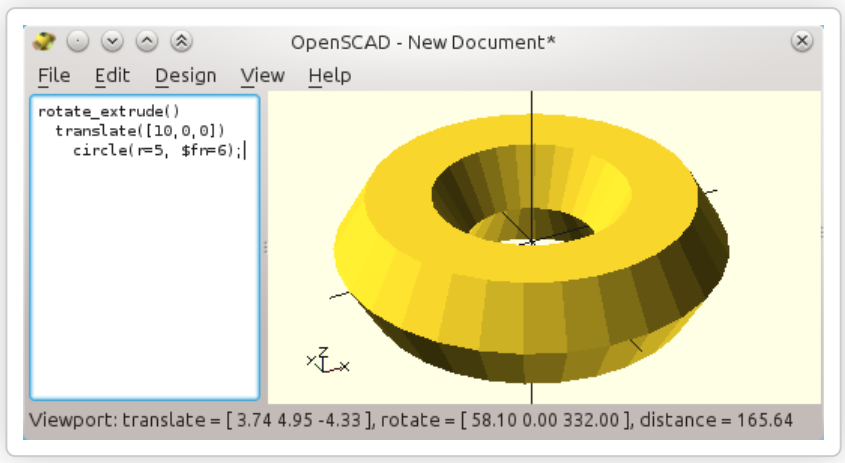
Here we will "raise" our former 2D shape vertically and make it a 3D shape, here is what we write:

```
linear_extrude(height=30)
{
  minkowski()
  {
    union()
    {
      square([20,20]);
      translate([20,10]) circle(r=6);
    }
    circle(r=4);
  }
}
```



3D linear extrusion of the former 2D shape

Contrary to the linear extrusion, the `rotate_extrude( )` operator can be used to create a "ring" out of a 2D profile. This is how we extrude a rough circle into a torus (note how the circle must be translated out of the origin to do so!).

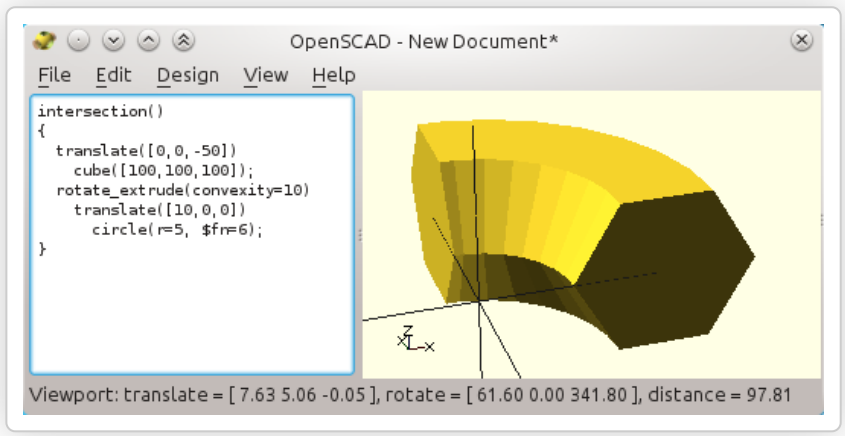


Rotational extrusion of a basic 2D hexagon (a 6-segment circle)

For the most curious readers, these extrusion operators have trickier optional properties (see the official documentation for more on this). If you cut the toroid by an orthogonal plane you will find the 6-segment primitive circle (hence the rough look).

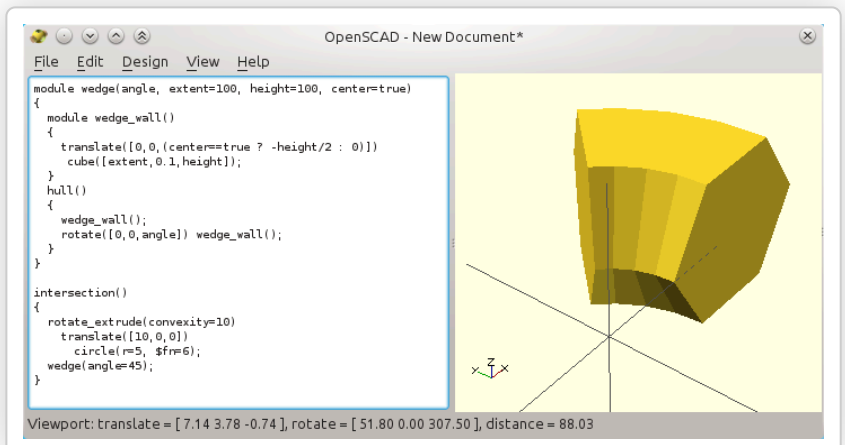
One annoying thing with the "rotate_extrude( )" is that it does not allow partial rotations. It will always "close the loop".

So when you need only 1/4th of a turn (i.e. a partial 90° rotation), say, then the best way is to intersect the shape with a cube, like this:



Easy 90° slice of the torus, with an intersection of the former torus and a big cube that stands in the positive (X,Y,Z) spatial quadrant.

Now, for any angles smaller than 90°, we can create a "wedge" shape by means of a convex hull around two very thin slices (almost flat cuboids), where one of them is rotated by the expected angle (see, by the way how modules can be defined within modules).



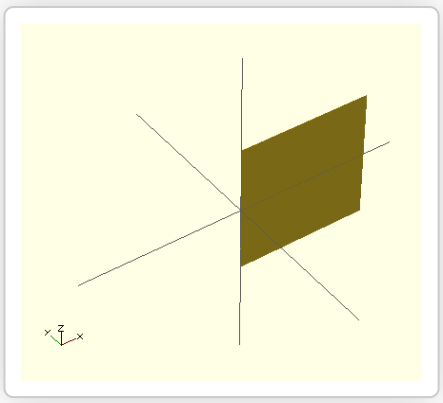
A 45° "slice" of the initial torus, but it works only with for small wedges.

The "wedge_wall" thickness should be chosen below the 3D printer resolution, e.g. as low as 0.02 mm. But take care as too small a value and Openscad may just "skip" the shape!).

You may have seen the use of the weird expression `(center==true ? -height/2 : 0)`. This is a compact form of a conditional, usually barely readable: "if center is true, then replace the expression by the value -height/2 else use the value 0". This is a short way to code a condition, which is used here to shift the part downwards by half the height in order to center it, like it is done with cylinders.

We did not use a more usual "if" test, as in the recursive mug. It would make life harder as "if" works with shapes, and not operators: the "if" condition cannot apply on the presence of the "translate" operation itself, but only on whole shapes. The "?" shorter version applies only on mathematical expressions, suitable here. All in all, in some case we will ask for a translation of (0,0,0), which works!

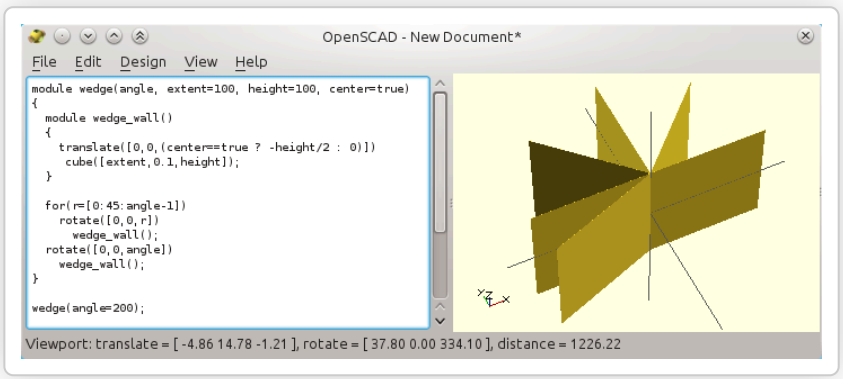
Anyhow. The above wedge design fails when the angle is larger than 90°, as the hull will flatten the two-wall wedge (see the animation below).



A convex hull between the first and
last "wall" fails with any large angles!
We will see Openscad animation later.

To create a pseudo-wedge that really can span 360°, we can use a succession of hulls no bigger than 45°, and that are progressively rotated around the Z axis.

So using a loop is very natural to generate the intermediate walls, spaced here at 45° of each other.
Still, we need to add the last wall "manually" as it may not fall on a multiple of 45° as below.



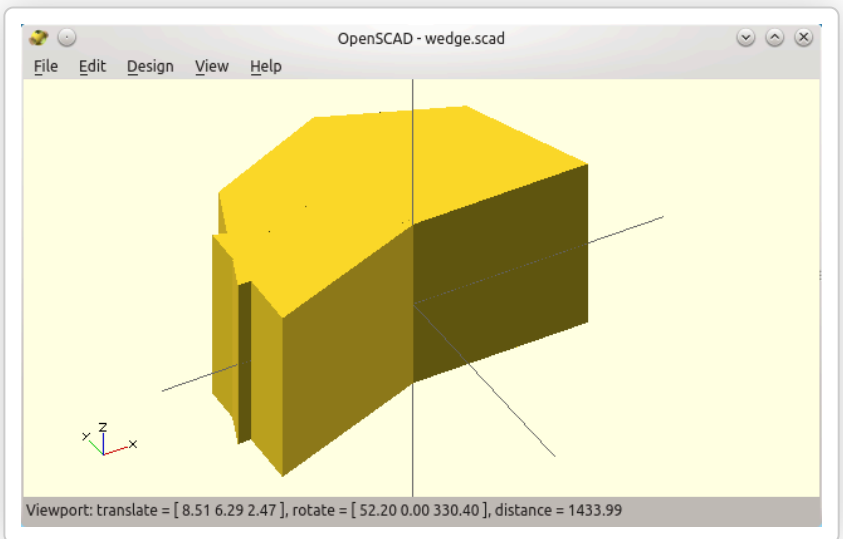
Angularly-spaced thin wall to help building the required concave wedge.

```
module wedge(angle, extent=100, height=100, center=true)
{
  module wedge_wall()
  {
    translate([0,0,(center==true ? -height/2 : 0)])
      cube([extent,0.1,height]);
  }

  for(r=[0:45:angle-1])
    rotate([0,0,r])
      wedge_wall();
  rotate([0,0,angle])
    wedge_wall();
}

wedge(angle=200);
```

Now, we want a shell somehow like a hull around this skeleton...

But a regular `hull( )` around the thin walls would fail badly, as it would "fill" also straight from the first wall to the last wall. The required concavity made by the first and last walls would not be respected. Sure, as the operator builds a *convex* hull. A concave hull probably has no meaning anyway, we must build it by hand.

So what's the trick? The expected concave wedge shape can be made by unions of successive hulls!



Here is our first attempt at a potential "concave" wedge.
It works but it makes a dirty source code (so far).

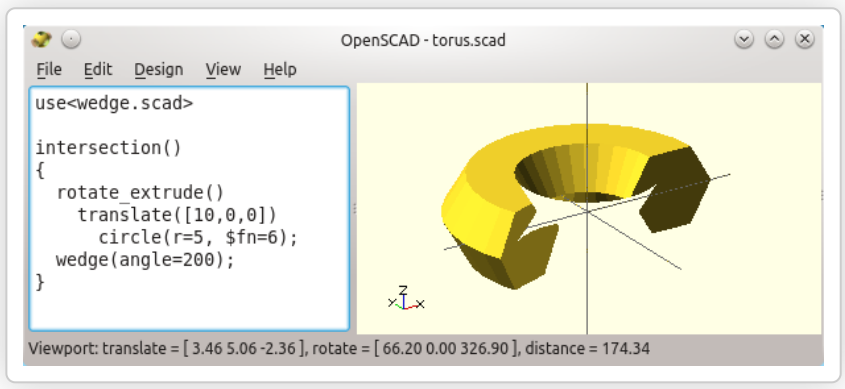
This is how we did it. We also introduces the `min(v1,v2,v3..)` and `max()` mathematical operators (weirdly, they do not take a vector as argument but a list of variables).

```
module wedge(angle, extent=100, height=100, center=true)
```

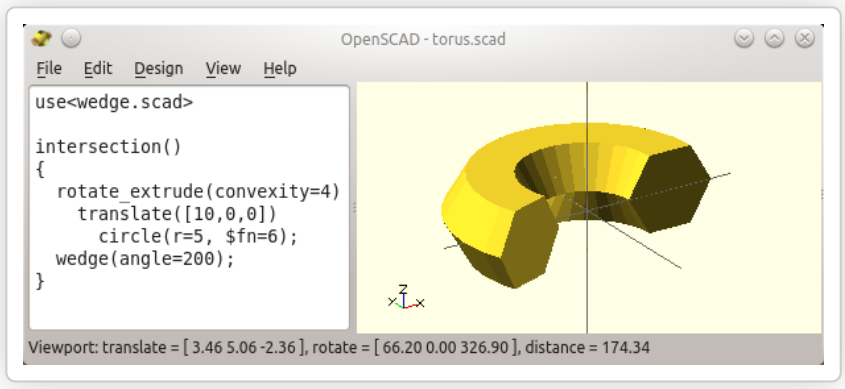
```
{
  module wedge_wall()
  {
    translate([0,0,(center==true ? -height/2 : 0)])
      cube([extent,0.1,height]);
  }

  for(r=[0:45:angle-45-1])
  {
      hull()
      {
        rotate([0,0,r]) wedge_wall();
        rotate([0,0,min(angle,r+45)]) wedge_wall();
      }
  }
  hull()
  {
    rotate([0,0,max(0,angle-45)]) wedge_wall();
    rotate([0,0,angle]) wedge_wall();
  }
}

wedge(angle=200);
```

The `min` and `max` are required to avoid overshooting the expected wedge `angle`.

Here are the result of the intersection of our wedge and the initial torus, that finally does it.



It works! The parametric wedge "sliced" the torus the way we wanted.
Note the "missing" part? It is only a rendering issue, see below!



Here is why there is a "convexity" parameter in the rotate_extrude(), and a few other Openscad functions. It helps the renderer by telling to look deeper in the intersection of objects.

Well... OK, it works... but the source code got much polluted...

It can get cleaner with the beautiful and somehow forgotten / unusual concept of `children` in Openscad. And that will make yet another part in this introduction to Openscad (to come!)

Posted by Jeremie Francois at 8:57 PM

Labels: review, software

No comments:

Post a Comment

To leave a comment, click the button below to sign in with Google.

SIGN IN WITH GOOGLE

About Me

Jeremie Francois

View my complete profile

Follow by RSS

Posts

Comments

Blog Archive

► 2017 (3)
► 2016 (7)
► 2015 (9)
▼ 2014 (18)
▼ Nov 2014 (3)
How to use Openscad (4): children and advanced topics
How to use Openscad (3): iterations, extrusions an...
Fortune 500 teller? 3D printing trends, reports, a...
► Sep 2014 (2)
► Aug 2014 (2)
► Jul 2014 (1)
► Jun 2014 (2)
► May 2014 (2)
► Mar 2014 (2)
► Jan 2014 (4)
► 2013 (14)
► 2012 (29)

This work is licensed CC BY-SA 4.0. Awesome Inc. theme. Powered by Blogger.